

RAG-Guard

Evaluating and Improving the Reliability of Retrieval-Augmented Generation Systems

Domain 03 — Artificial Intelligence, Machine Learning & LLMs | Pre-Interview Competitive Assessment

1. Executive Summary

RAG-Guard is an experimental evaluation framework for Retrieval-Augmented Generation (RAG) reliability, built and tested against a controlled corpus of official Python 3.11 and 3.12 documentation. Rather than building another chatbot, the objective was to measure how reliably a RAG system retrieves supporting evidence and generates grounded answers, identify a genuine failure pattern, and test whether a targeted improvement actually helps.

The central finding was not a simple retrieval problem. Investigation revealed that several baseline answers which appeared correct were not actually grounded in retrieved evidence — the model was silently falling back on its own training knowledge when retrieval quality was weak, despite an explicit instruction not to do so. A stricter, evidence-quoting generation prompt eliminated all detected instances of this behavior (4 → 0 faithfulness violations), but at a measurable cost: overall correctness fell from 88% to 79%, because the system became more conservative and abstained on two borderline multi-part questions rather than partially answer with incomplete evidence.

This trade-off — lower raw correctness in exchange for zero undetected hallucination — is reported as the project's central result, rather than a flattering single number.

2. Problem & AI Opportunity

Large language models can generate confident, plausible answers even when they do not have reliable information to support them. Retrieval-Augmented Generation attempts to address this by grounding generation in retrieved evidence, but RAG introduces its own failure modes: the retriever may fail to find relevant evidence, may retrieve irrelevant or distracting context, or the generator may ignore retrieved evidence altogether and answer from its own parametric knowledge regardless of what it was given.

The problem this project investigates is therefore not "how do we build RAG" but "how do we know whether a RAG system is actually reliable, and what should we change when it fails?" This matters because a RAG system that silently answers from memory when retrieval fails is more dangerous than one that fails visibly — it looks reliable in casual testing while being fundamentally ungrounded, which is precisely the kind of failure that would go undetected without deliberate evaluation.

User: a developer or technical user querying a knowledge-intensive assistant. Input: a natural-language question. Output: an evidence-grounded answer, or an explicit statement that the evidence is insufficient. Failure cost: a confidently wrong or ungrounded answer can lead a user to act on unsupported information.

3. Landscape & Alternatives

Several retrieval and generation approaches were considered before implementation:

Approach	Considered?	Decision
Vector (embedding) retrieval	Yes	Used as the baseline retrieval method
BM25 keyword retrieval	Yes	Not implemented — deferred in favor of a tighter, fully-executed scope; noted as a likely fix for at least one observed failure (see Section 9)
Hybrid retrieval	Yes	Deferred for the same reason
Reranking	Yes	Deferred; the eventual improvement targeted generation, not retrieval, once evidence pointed there
Evidence-aware / quote-forced generation	Yes	Implemented as the final improvement, based on baseline failure analysis

The decision to scope tightly — one retrieval method, a small hand-verified evaluation set, one improvement chosen after seeing evidence rather than planned in advance — was a deliberate response to the project's 3-day constraint. A narrow, fully-executed experiment was judged preferable to a broad, partially-executed one.

4. AI Thesis / Hypothesis

Initial hypothesis: baseline vector RAG will retrieve relevant evidence for most direct and paraphrased questions, but reliability will decrease for unanswerable, distractor, and multi-document questions. A targeted evidence-aware improvement should reduce unsupported answers.

This hypothesis was partially supported and partially wrong in an informative way: retrieval was substantially better than initial automated measurement suggested (see Section 8), and the dominant baseline failure mode was not retrieval quality at all, but the generation model's willingness to answer from outside knowledge when retrieved evidence was present but imperfect.

5. Data & Corpus Strategy

Corpus: 19 official documentation pages each from Python 3.11 and 3.12 (38 source documents total), covering `asyncio`, `collections`, `json`, `os`, `pathlib`, `re`, tutorial error-handling pages, and the `whatsnew` release notes for both versions. This corpus was deliberately chosen because it contains real, officially documented version differences (e.g. `distutils` removed in 3.12, PEP 701 f-string changes) rather than artificially manufactured contradictions — this was judged important for constructing honest version-sensitive and multi-document evaluation questions.

Chunking strategy: paragraph-aware splitting targeting approximately 300 words per chunk, preserving code fences and section headings intact. Paragraphs shorter than 15 words were merged forward to avoid orphaned section-header chunks, and oversized bullet-list blocks were split at bullet boundaries to avoid one chunk spanning many unrelated list items. One paragraph of overlap was carried between consecutive chunks on the same page. This produced 1,254 chunks (613 from Python 3.11, 641 from Python 3.12), with a word-count range of 71–605 and an average of 275 words per chunk.

A known limitation of this approach: because chunking is performed independently per page and per version, the same chunk index (e.g. chunk 022) can contain different content between the 3.11 and 3.12 copies of a page, since chunk boundaries drift with small textual differences. This was discovered during failure analysis (Section 9) and is noted as a methodological caveat.

6. Architecture & Methodology

The pipeline consists of three stages, implemented as separate scripts for clarity and reproducibility:

1. Chunking (`chunker.py`) — described in Section 5.

2. Embedding & storage (`embed_and_store.py`) — each chunk is embedded using `all-MiniLM-L6-v2` (sentence-transformers) and stored in a persistent ChromaDB collection with version, source, and title as filterable metadata.

3. Retrieval & generation (`generate.py`) — for each question, the top-3 most similar chunks are retrieved via cosine similarity. Two LLM calls are made per question using `openai/gpt-oss-120b` (via the Groq API): a no-retrieval control (question only, no context) and a RAG-grounded call (question plus retrieved context).

The no-retrieval control was included specifically to detect parametric leakage — cases where the model's answer derives from its own training knowledge of Python rather than from the supplied context. This proved essential to the project's central finding (Section 9).

7. Baseline & Experiments

Baseline configuration: vector retrieval, top-K = 3, no reranking, a system prompt instructing the model to answer only from provided context and to state explicitly when context was insufficient.

This baseline was run once against the full frozen evaluation set (Section 8) and never modified after results were observed, consistent with the project's freeze discipline.

8. Evaluation Design

A set of 24 questions was hand-written and frozen before baseline evaluation, with every expected_evidence chunk_id manually verified against the actual corpus text (not generated or assumed). Six categories were used to target distinct failure modes:

Category	Count	What it tests
Direct	5	Baseline retrieval/generation competence
Paraphrased	5	Semantic retrieval beyond keyword overlap
Multi-document	4	Retrieval spanning more than one chunk/page
Version-sensitive	4	Correctly distinguishing 3.11 vs 3.12 facts
Unanswerable	4	Correct abstention when evidence is absent
Distractor	2	Resisting hallucination from related-but-irrelevant context

Primary metrics: retrieval hit rate (whether an expected evidence chunk appeared in the top-3 retrieved chunks), answer correctness (manually graded against expected_answer), faithfulness (whether the answer's claims were actually supported by the retrieved text, not just correct by coincidence), and abstention correctness for unanswerable/distractor questions.

Note on retrieval hit rate measurement: an initial automated exact-chunk-ID match produced a misleadingly low hit rate (44%), because the ground-truth file listed only one valid evidence chunk per question when the corpus in several cases contained multiple chunks that independently supported the same fact (e.g. a summary list and a dedicated detail section both covering the same deprecation). After manually reading the full text of every retrieved-but-unlisted chunk, the corrected retrieval hit rate was found to be $15/18 = 83\%$ among answerable questions. This measurement correction is itself reported as a methodological finding: naive exact-ID matching can understate real retrieval quality when a corpus contains redundant coverage of the same fact.

9. Failure Analysis

Baseline correctness was 21/24 (88%). However, closer inspection — comparing each RAG-grounded answer against its no-retrieval control counterpart — revealed that several "correct" answers were not actually grounded in the retrieved context:

Parametric leakage (3 cases: re.search vs re.match purpose, the try/else clause, and the purpose of ExceptionGroup): in each case, the RAG answer closely matched the no-retrieval control answer in content, strongly indicating the model answered from its own training knowledge rather than the retrieved context, despite an explicit system-prompt instruction not to do so.

One mixed case (distutils version-sensitivity question): the answer correctly used genuine retrieved evidence for the Python 3.12 removal fact, but the claim about Python 3.11's "pending removal" status was not actually supported by any retrieved chunk — a partial leak within an otherwise-correct answer.

Separately, three questions failed due to genuine retrieval misses, where none of the top-3 retrieved chunks contained the needed evidence, and the system correctly abstained rather than hallucinate: a multi-document question requiring both a general asyncio definition and a TaskGroup-specific chunk (only the former was retrieved), a version-sensitive question about a specific PEP 701 f-string restriction, and an abstract cross-document synthesis question that proved poorly suited to chunk-level retrieval in general.

All 6 unanswerable and distractor questions were correctly abstained on in the baseline, with zero hallucinations — a genuine strength of the baseline system worth preserving.

10. Iteration: The Improvement

The dominant, most actionable baseline failure was parametric leakage, not retrieval quality — a generation-side grounding discipline problem. The improvement was chosen after this evidence was collected, not planned in advance.

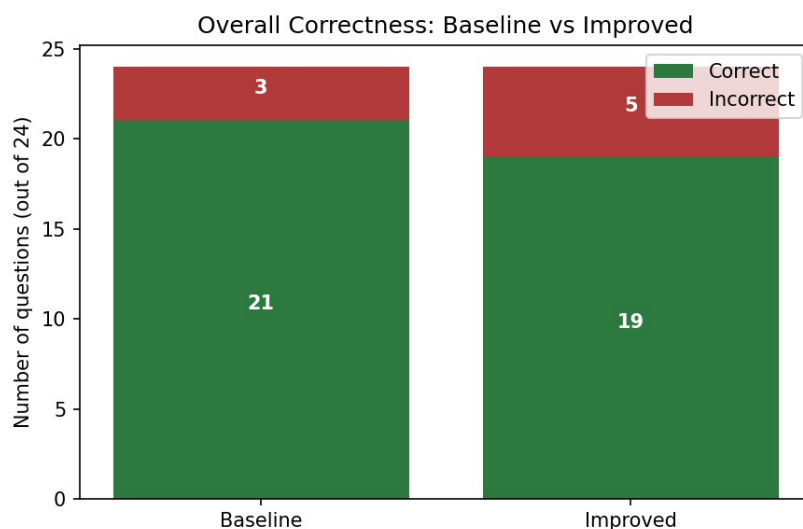
The generation system prompt was rewritten to require the model to first quote the exact supporting sentence(s) from the retrieved context under a "Supporting evidence:" heading before producing a final answer, and to abstain explicitly if no genuine supporting quote could be found — making it structurally harder for the model to substitute outside knowledge for retrieved evidence.

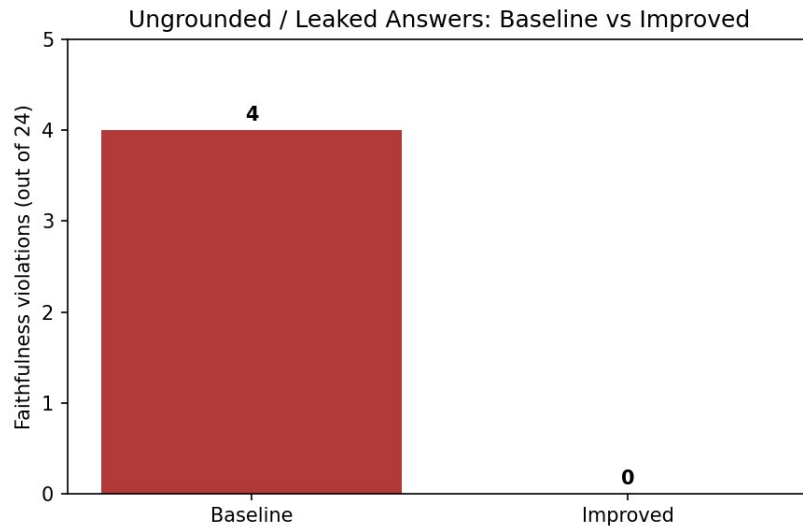
This was verified as a genuine architectural fix rather than a superficial prompt tweak: for the two previously-leaking questions that remained answerable after the change, the supporting quotes produced were checked against the actual chunk text and confirmed to be verbatim, not fabricated.

11. Final Outcome

The same 24 questions were re-run, unmodified, against the improved system.

Metric	Baseline	Improved
Overall correctness	21/24 (88%)	19/24 (79%)
Faithfulness violations (ungrounded/leaked answers)	4	0
Retrieval hit rate (corrected)	15/18 (83%)	unchanged (retrieval untouched)





The improvement fully achieved its target: both previously-leaking questions that remained answerable were fixed and verified as genuinely grounded. However, it introduced two regressions, each interpretable rather than simply negative:

Question 15 (distutils, version-sensitive): baseline had answered this "correctly" partly via leakage (see Section 9). The improved system correctly recognized it could not fully support both halves of this two-part question from retrieved evidence alone, and abstained entirely rather than repeat the partial leak. This is a corrected behavior, not a new bug.

Question 9 (ExceptionGroup): this question was answered correctly and genuinely grounded in an earlier isolated test of the improved prompt, then abstained on an identical question with identical retrieved context in the full re-run. This is evidence of run-to-run non-determinism in the underlying hosted LLM despite temperature=0.0, and is reported honestly as an observed limitation rather than smoothed over.

Net result: the improvement traded roughly two points of raw correctness for the complete elimination of every detected instance of silent, ungrounded answering. For a system whose stated goal is reliability rather than raw answer rate, this trade-off is judged a genuine improvement, though a real one with a real cost — not an unqualified win.

12. Limitations & Responsible AI

Sample size: 24 questions (as few as 2 per category) is sufficient for pattern-level diagnostic analysis but not for statistically generalizable performance claims. Category-level results should be read as evidence of specific failure modes, not as population-level accuracy estimates.

Embedding model: all-MiniLM-L6-v2 is a small, general-purpose model used without domain fine-tuning; retrieved-chunk cosine distances were moderate (0.31–0.40) rather than tight, consistent with this choice.

LLM non-determinism: at least one case of answer variance was observed between runs on an identical question with identical retrieved context, despite temperature=0.0 — a known characteristic of some hosted inference stacks.

Chunk-index drift across versions: the same chunk index number does not correspond to the same content between the 3.11 and 3.12 copies of a page, since chunking is performed independently per document.

Ground-truth coverage: the initial evaluation file listed only one canonical evidence chunk per question in several cases, when the corpus in fact contained multiple valid supporting chunks — corrected during analysis but worth noting as a data-preparation lesson.

13. Next Development Stage

Priority next steps, in order of expected impact: (1) address the two multi-document/version-sensitive retrieval misses directly, likely via a lightweight keyword (BM25) fallback specifically for exact technical terms like PEP numbers, which a pure embedding model is prone to miss; (2) investigate a partial-answer mode that discloses which half of a compound question lacks support, rather than abstaining entirely, to recover the two questions traded away in Section 11 without reintroducing leakage; (3) expand the evaluation set with more repetitions per category to quantify the observed LLM non-determinism rather than rely on a single anecdotal instance.

14. Tool & Assistance Declaration

A full declaration of models, libraries, and AI assistance used during this project is provided separately in 06_DECLARATION/AI_EXTERNAL_TOOLS.md, including what was verified personally versus AI-assisted.